1주차 코드리뷰

브루트 포스

2020111414 최지은

브루트 포스 (Brute Force)

의미

- 완전탐색 알고리즘
- 모든 경우의 수를 탐색하면서 요구조건에 충족되는 결과만 가져오는 알고리즘
- 해가 존재할 것으로 예상되는 모든 영역을 전체 탐색

특징

장점) 구현이 쉽다, 100%의 확률로 정답 출력

단점) 모든 경우의 수를 계산하기 때문에 시간적인 측면에서 비효율적

예시

4자리 숫자로 된 핸드폰의 암호 (0000, 0001, 0002... 9999), 10의 약수의 합을 구하기

```
int sum = 0;
  for(int i = 1; i <= n; i = i + 1)
    if(n % i == 0)
      sum = sum + i;
print sum on the screen;
```

-> 즉, 입력 값의 범위가 작은 알고리즘을 구현할 때 적합한 방법

코드리뷰
#1.연산자 끼워넣기

```cpp
    int n;
    int num[11];
    int op[4];

    int max_value = -1000000000;
    int min_value = 1000000000;
```

```cpp
int main()
{
    cin >> n;

    for (int i = 0; i < n; i++)
        cin >> num[i];
    for (int i = 0; i < 4; i++)
        cin >> op[i];

    calValue(num[0], 1);

    cout << max_value << '₩n'
        << min_value << '₩n';

    return 0;
}
```

```cpp
void calValue(int value, int cnt)
{
    if (cnt == n)
    {
        if (value > max_value)
            max_value = value;
        if (value < min_value)
            min_value = value;
    }

    for (int i = 0; i < 4; i++)
    {
        if (op[i] > 0)
        {
            op[i]--;
            if (i == 0)
                calValue(value + num[cnt], cnt + 1);
            else if (i == 1)
                calValue(value - num[cnt], cnt + 1);
            else if (i == 2)
                calValue(value * num[cnt], cnt + 1);
            else if (i == 3)
                calValue(value / num[cnt], cnt + 1);

            op[i]++;
        }
    }
}
```

재귀 함수 호출 순서를 바꿔 동작 가능하게 함

→ 재귀함수를 빠져나오면 호출하기 이전 값과 동일함

코드리뷰
#2.파이프 옮기기1

```cpp
const int MAXSIZE = 16;

int n;
int cnt = 0;
int home[16][16];

```

```cpp
int main()
{
    cin >> n;

    int s;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cin >> s;
            home[i][j] = s;
        }
    }

    movePipe(0, 0, 1);

    cout << cnt;

    return 0;
}
```

```cpp
void movePipe(int state, int r, int c) {

    if (r == n - 1 && c == n - 1)
    {
        cnt++;
        return;
    }

    bool canMoveH = c + 1 < n && home[r][c + 1] == 0;
    bool canMoveV = r + 1 < n && home[r + 1][c] == 0;

    if (canMoveH && state != 1)
        movePipe(0, r, c + 1);

    if (canMoveV && state != 0)
        movePipe(1, r + 1, c);


    if (canMoveH && canMoveV && home[r + 1][c + 1] == 0)
        movePipe(2, r + 1, c + 1);
}
```

불가능한 경우를 가지치기 하면서 재귀함수 호출

1. 가로 -> 세로, 세로 -> 가로로 회전 X
2. 파이프의 끝이 범위를 벗어나거나 벽 위에 있을 경우 X
3. 마지막으로 파이프의 방향이 대각선인 경우 가로와 세로인 경우와 다르게 확인해야 할 벽이 두 군데 더 존재

THANK YOU

감사합니다